

Java Lab Record: Longest Substring Without Repeating Characters

Objective / Aim

To write a Java program that finds the **longest substring without repeating characters** in a given string, using **static methods** and **String handling**, and displays both its length and the substring itself.

Problem Statement

Write a program to find the longest substring without repeating characters.

Input: abcabcbb Output: 3 String: abc
Input: pwwkew Output: 3 String: wke

Note: pwke is **not** a substring, it is a **subsequence**. A substring must be made of characters that sit **next to each other** in the original string; a subsequence may skip characters.

System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

Algorithm / Logic Description

1. Create a class named LongestSubstringWithoutRepeatingCharacters.
2. Create a static method findLongestSubstring(String input).
3. Use a HashSet<Character> to store the characters of the current

- window (the part of the string with no repeated characters).
4. Use two pointers:
 - start -> beginning of the current window
 - end -> current character being checked
 5. Keep track of longestStart and longestLength of the best window.
 6. For every character at position 'end':
 - a. While the character is already in the set, remove the character at 'start' from the set and move 'start' forward.
 - b. Add the character to the set.
 - c. Window length = end - start + 1.
 - d. If this length is greater than longestLength, update longestLength and longestStart.
 7. Return input.substring(longestStart, longestStart + longestLength).
 8. In main(), read a string from the user, call the method, and display the length and the substring.

Real-time Application

Text analysis, data compression, username / password uniqueness checks, and detecting repeated patterns in streaming data.

LongestSubstringWithoutRepeatingCharacters.java

Source Code

```
package beyond;

import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class LongestSubstringWithoutRepeatingCharacters {

    public static String findLongestSubstring(String input) {
        // A HashSet stores the characters in the current window and checks
        for repeats quickly.
        Set<Character> characters = new HashSet<>();
        int start = 0;
        int longestStart = 0;
        int longestLength = 0;

        for (int end = 0; end < input.length(); end++) {
            char current = input.charAt(end);

            // Move the start forward until the repeated character leaves the
            window.
            while (characters.contains(current)) {
```

```

        characters.remove(input.charAt(start));
        start++;
    }

    characters.add(current);
    int currentLength = end - start + 1;

    // Keep the first substring when multiple substrings have the
    same length.
    if (currentLength > longestLength) {
        longestLength = currentLength;
        longestStart = start;
    }
}

return input.substring(longestStart, longestStart + longestLength);
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String input = scanner.nextLine();

    String longestSubstring = findLongestSubstring(input);
    System.out.println("Length: " + longestSubstring.length());
    System.out.println("Substring: " + longestSubstring);

    scanner.close();
}
}

```

1. Concept and Exception in Simple Words

Concepts used

Static method -> findLongestSubstring() is declared 'static', so it belongs to the CLASS, not to an object. main() calls it directly without writing 'new' first.

String handling -> charAt(i) reads one character, length() gives the size, substring(a, b) cuts out a part of the string.

Sliding window -> two pointers (start and end) mark a window of characters; the window grows on the right and shrinks from the left whenever a repeat appears.

HashSet -> stores the window's characters and answers "is this character already here?" very quickly.

Note: The topic list also mentions the `this` reference and the `final` keyword. This particular program does **not** use either one: it has no instance fields (so there is nothing for `this` to point to) and no `final` variables. They could be added, for example `final String input`, but the code works without them.

Substring vs Subsequence

String: pwwkew
 Substring "wke" -> characters are next to each other (valid)
 Subsequence "pwke" -> skips the second 'w' in the middle (NOT a substring)

Exception in simple words

No exception is thrown or caught in this program, and normal inputs run cleanly. A few edge cases are worth knowing:

Situation	What happens
Empty string (just press Enter)	Loop never runs, <code>substring(0, 0)</code> returns "", so it prints Length: 0 and an empty substring. No exception.
All same characters, e.g. bbbb	Window keeps shrinking to 1, result is b with Length: 1.
No input available at all (e.g. input stream closed)	<code>scanner.nextLine()</code> throws <code>NoSuchElementException</code> (unchecked).
input is null (only possible if another class passes null)	<code>input.length()</code> would throw <code>NullPointerException</code> . Not possible from this <code>main()</code> .

2. Step-wise Execution

1. Program starts in `main()`. A Scanner object is created.
2. User enters a string, e.g. "pwwkew".
3. `findLongestSubstring("pwwkew")` is called (static call, no object).
4. Inside the method:


```

characters = {} , start = 0, longestStart = 0, longestLength = 0

end=0  'p'  not in set -> add          set={p}          len=1
        1 > 0 -> longestLength=1, longestStart=0
end=1  'w'  not in set -> add          set={p,w}         len=2
        2 > 1 -> longestLength=2, longestStart=0
end=2  'w'  IN set:
        remove 'p' (start=0), start=1
        'w' still in set: remove 'w' (start=1), start=2
        add 'w'          set={w}          len=1
        1 > 2 ? no change
end=3  'k'  not in set -> add          set={w,k}         len=2
        2 > 2 ? no change
end=4  'e'  not in set -> add          set={w,k,e}        len=3
        3 > 2 -> longestLength=3, longestStart=2
end=5  'w'  IN set:
        remove 'w' (start=2), start=3
        add 'w'          set={k,e,w}        len=3
        3 > 3 ? no change (first one is kept)
      
```
5. Return `substring(2, 2+3) = substring(2, 5) = "wke"`.
6. `main()` prints: Length: 3 and Substring: wke
7. Scanner is closed.

Second dry run: input "abcabcbb"

```

end=0 'a' add          {a}          len=1   best=1 ("a")
end=1 'b' add          {a,b}        len=2   best=2 ("ab")
end=2 'c' add          {a,b,c}      len=3   best=3 ("abc", start=0)
end=3 'a' repeat -> remove 'a', start=1; add 'a' {b,c,a} len=3 (no change)
end=4 'b' repeat -> remove 'b', start=2; add 'b' {c,a,b} len=3 (no change)
end=5 'c' repeat -> remove 'c', start=3; add 'c' {a,b,c} len=3 (no change)
end=6 'b' repeat -> remove 'a', remove 'b', start=5; add 'b' {c,b} len=2
end=7 'b' repeat -> remove 'c', remove 'b', start=7; add 'b' {b} len=1
Result: substring(0, 3) = "abc" -> Length: 3

```

3. Algorithm

START

1. Create Scanner object
2. Read input string from user
3. CALL findLongestSubstring(input):
 - a. Create empty HashSet characters
 - b. SET start = 0, longestStart = 0, longestLength = 0
 - c. FOR end = 0 TO input.length() - 1:
 - i. current = input.charAt(end)
 - ii. WHILE characters contains current:

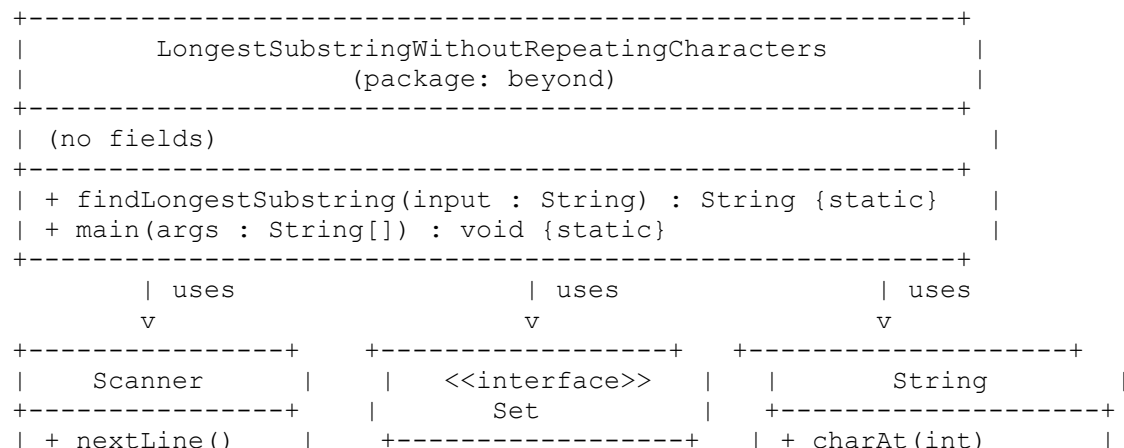

```
remove input.charAt(start) from characters
start = start + 1
```
 - iii. add current to characters
 - iv. currentLength = end - start + 1
 - v. IF currentLength > longestLength:

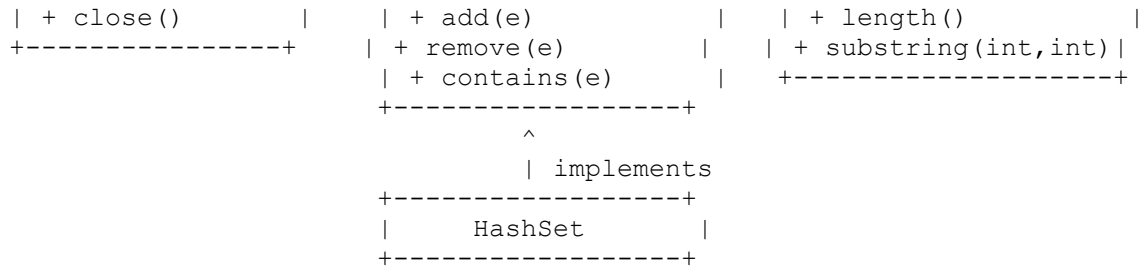

```
longestLength = currentLength
longestStart = start
```
 - d. RETURN input.substring(longestStart, longestStart + longestLength)
4. PRINT "Length: " + result.length()
5. PRINT "Substring: " + result
6. Close Scanner

STOP

Time complexity : $O(n)$ - each character is added and removed at most once
 Space complexity : $O(k)$ - k = number of distinct characters in the window

4. UML Class Diagram





Legend: + public, {static} belongs to the class itself.

Compilation and Execution

Folder structure:

```

project/
|-- beyond/
    |-- LongestSubstringWithoutRepeatingCharacters.java

```

Compile:

```
javac beyond/LongestSubstringWithoutRepeatingCharacters.java
```

Run:

```
java beyond.LongestSubstringWithoutRepeatingCharacters
```

Sample Output

```

Enter a string: abcabcb
Length: 3
Substring: abc

```

```

Enter a string: pwwkew
Length: 3
Substring: wke

```

```

Enter a string: bbbb
Length: 1
Substring: b

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basic java> C:\Program Files\Java\jdk-20.0.1\bin\java.exe -enable-preview -xx:rsShowObjectDetailsInExceptionMessages -cp C:\Users\Admin\OneDrive\Documents\basic java\bin\ beyond.LongestSubstringWithoutRepeatingCharacters
Enter a string: programming
Length: 5
Substring: ogram

```

Result

The Longest Substring Without Repeating Characters program was successfully implemented in Java using static methods and String handling. The program uses a sliding window with a HashSet to find the longest substring that contains no repeated characters, correctly producing length 3 with substring "abc" for the input "abcabcb" and length 3 with substring "wke" for the input "pwwkew", while running in a single pass over the string.